

RED TEAM LEARNING PATH

MODULE 17

WINDOWS PRIVILEGE ESCALATION

Access tokens, ACLs, services, scheduled tasks, autoruns, credential exposure, Sysinternals, automation & remediation

Authorized-lab only Hands-on dilakukan pada disposable Windows VM milik sendiri atau training platform yang memberi izin. Local lab bundled sengaja tidak membuat service/task persistence nyata; ia hanya mensimulasikan privileged execution boundary yang reversible.

Omarchy / Arch Linux Edition

Project: VaxXinE/red-teaming | Version 1.0 | 05 Oktober 2026

Field	Value
Primary host	Omarchy / Arch Linux + disposable Windows 11 Pro VM
Prerequisite	Modules 00-16
Core tools	PowerShell, whoami, icacis/Get-Acl, CIM/WMI, Sysinternals AccessChk/Autoruns/Procmon, WinPEAS
Guided practice	15 labs + capstone
Local exploitation model	Reversible writable-script + elevated runner simulation; no real service/task registration
Next module	18 - Active Directory Fundamentals

Module map

Phase	Outcome
Context	Understand token, SID, privileges, integrity level, UAC and SYSTEM.
Manual enum	Users/groups, services, tasks, ACLs, autoruns, paths, configs, network/processes.
Hypothesis	Find a privileged executor that trusts attacker-controlled config/file/path/permission.
Controlled proof	Demonstrate one privilege-boundary failure inside the lab with minimum side effect.
Compare & report	Use AccessChk/Autoruns/Procmon/WinPEAS after manual reasoning; document root cause, telemetry, cleanup, remediation.

1. Learning Objectives & Safety Boundary

- [] Membaca Windows security context: SID, groups, token privileges, integrity level, UAC, service accounts dan LocalSystem.
- [] Melakukan manual enumeration sebelum automated tooling.
- [] Menganalisis file/registry/service/task ACL sebagai trust boundary, bukan sekadar mencari string "writable".
- [] Memahami common Windows privesc classes: service misconfiguration, writable privileged path, scheduled task action, autorun path, token privilege, credentials/config, DLL/search-path behavior, dan vulnerable driver/kernel sebagai last resort.
- [] Membedakan misconfiguration presence dari exploitability dan membuktikan impact dengan minimum sufficient proof.
- [] Menggunakan Sysinternals/WinPEAS sebagai evidence amplifier setelah hipotesis terbentuk.
- [] Menulis remediation yang memperbaiki permission/trust model, bukan cuma menghapus PoC.

Windows VM di Omarchy Omarchy menyediakan Windows 11 Pro VM berbasis KVM/Docker, dengan port VM dibind ke localhost dan compose file dimiliki root. Gunakan VM disposable/snapshot untuk latihan privilege escalation.

2. Mental Model: Token Meets Security Descriptor

Windows privilege escalation paling mudah dipahami sebagai pertemuan dua sisi: proses membawa access token; object membawa security descriptor. Windows melakukan access check ketika proses mencoba mengakses file, registry key, service object, process, task, atau securable object lain.

```
PROCESS / THREAD
  access token
  - user SID
  - group SIDs
  - privileges
  - integrity level
    |
    v access check
SECURABLE OBJECT
  security descriptor
  - owner
  - DACL / ACEs
  - SACL / mandatory label
```

Primitive	Question to ask
Token	Siapa gue? Groups apa? Privileges apa? Medium/high/system integrity?
File/Directory ACL	Bisa write/modify/replace? Parent directory writable? Inheritance bagaimana?
Service object	Bisa change config/start/stop/write DACL? Service berjalan sebagai siapa?
Scheduled task	Task berjalan sebagai siapa? Action executable/script writable?
Registry / autorun	Apakah privileged component membaca value/path yang bisa gue ubah?
Process / token	Ada privilege spesial atau impersonation boundary yang relevan?
Credentials	Ada secret reusable yang memberi konteks privilege lebih tinggi?

Finding = path, not artifact Writable file, enabled privilege, atau unquoted path bukan finding otomatis. Finding valid memerlukan attacker-controlled input + privileged consumer + feasible trigger + measurable impact.

3. First Five Minutes: Manual Enumeration

```
whoami /all
whoami /priv
whoami /groups

Get-ComputerInfo | Select WindowsProductName,WindowsVersion,OsBuildNumber,OsArchitecture,CsName
Get-LocalUser
Get-LocalGroupMember Administrators -ErrorAction SilentlyContinue
```

Simpan output terstruktur sebelum mulai perubahan apa pun. Access token berisi user SID, group SIDs dan privileges; `whoami /all` adalah baseline cepat untuk context tersebut.

```
powershell -ExecutionPolicy Bypass -File .\labs\workspace-init.ps1
powershell -ExecutionPolicy Bypass -File .\labs>manual-enum.ps1
```

Do not dump everything Jangan otomatis mengumpulkan SAM/LSASS/browser secrets/DPAPI material. Module ini fokus ke privilege boundary dan misconfiguration; credential dumping adalah scope berbeda dan risikonya jauh lebih tinggi.

4. Access Tokens, Integrity Levels & UAC

Standard user umumnya berjalan pada medium integrity. Elevated administrator process memakai high integrity, sedangkan banyak system services memakai system integrity. Mandatory Integrity Control dievaluasi selain DACL; lower-integrity process tidak otomatis boleh menulis ke higher-integrity object.

```
whoami /all
whoami /groups | findstr /i "Mandatory Label"
```

```
whoami /priv
```

Token privilege	Why it matters conceptually
SeImpersonatePrivilege	Impersonation boundary; sangat sensitif pada service contexts.
SeBackupPrivilege	Dapat membuka akses backup semantics terhadap protected data.
SeRestorePrivilege	Dapat menulis/restore object dengan semantics khusus.
SeTakeOwnershipPrivilege	Dapat mengubah ownership object lalu memengaruhi ACL path.
SeDebugPrivilege	Dapat membuka akses sangat luas ke proses lain ketika enabled/allowed.
SeLoadDriverPrivilege	Driver loading adalah high-risk kernel trust boundary.

Privilege present != exploit State (Enabled/Disabled), process type, OS build, service context, patch level dan usable primitive menentukan exploitability. Jangan langsung mencari exploit hanya karena privilege muncul.

5. File, Directory & Registry ACLs

ACL bugs sering menjadi akar Windows privilege escalation: privileged process menjalankan atau membaca sesuatu yang bisa dimodifikasi low-privilege user. Analisis juga parent directory dan inheritance.

```
Get-Acl C:\Path\To\Target | Format-List Path,Owner,AccessToString
icacls C:\Path\To\Target

# helper
.\labs\acl-audit.ps1 -Path C:\Path\To\Target
```

What permissions are interesting?

Right / pattern	Interpretation
F / FullControl	Can alter content/ACL/ownership depending object.
M / Modify	Usually enough to replace or modify file content.
W / Write	Need inspect exact ACE/object type; often security-relevant.
WDAC / WRITE_DAC	Can change DACL and potentially grant self access.
WO / WRITE_OWNER	Can change owner and then potentially alter permissions.
Writable parent	Even if file is protected, replacement may be possible through directory rights depending semantics.

Registry is securable too Registry keys have security descriptors and ACLs. A privileged service/autorun that reads executable path/config from a key writable by low-privilege users can create the same trust problem as a writable file.

6. Windows Services: The Highest-Value Configuration Surface

```
Get-CimInstance Win32_Service |
  Select Name,StartName,State,StartMode,PathName |
  Sort Name

.\labs\service-audit.ps1
```

For each interesting service, answer four questions: siapa account-nya, executable/config path-nya apa, siapa yang boleh mengubah service object, dan siapa yang boleh memodifikasi executable/path/config tersebut.

Service weakness	Why it can matter
Writable service binary/config	Low-priv user can influence code/data later executed by privileged service.
Weak service object DACL	SERVICE_CHANGE_CONFIG can let caller change executable configuration; Microsoft explicitly treats this right as admin-sensitive.
Unquoted service path	Only exploitable when parsing creates a candidate path AND attacker can write a relevant directory/file AND trigger the service.
Writable dependency / working directory	Privileged program may load user-controlled DLL/config/helper.
Dangerous service account	LocalSystem or privileged domain/service account increases impact.

AccessChk workflow

```
accesschk.exe -accepteula -uwcqv "BUILTIN\Users" *
accesschk.exe -accepteula -qvw C:\ProgramData\Vendor\AppData
```

Read the output, do not cargo-cult it AccessChk answers "what access exists"; lo masih harus map access -> privileged consumer -> trigger -> impact. SERVICE_CHANGE_CONFIG and writable binaries are separate primitives.

7. Scheduled Tasks

```
Get-ScheduledTask | Select TaskPath,TaskName,State,
@{n='RunAs';e={$_.Principal.UserId}},
@{n='RunLevel';e={$_.Principal.RunLevel}}
.\labs\task-audit.ps1
```

Task Scheduler runs tasks under a configured security context. A task is interesting when it runs elevated/high-privilege and references an action, script, working directory or config that a lower-privilege user can modify.

- [] RunAs account privileged?
- [] RunLevel Highest?
- [] Action path absolute?
- [] Action/script writable?
- [] Parent directory writable?
- [] Arguments reference writable config?
- [] Trigger actually occurs in engagement window?

Task existence is not a vulnerability A SYSTEM scheduled task is normal. The vulnerability is a broken trust edge such as low-privilege modification of its action or configuration.

8. Autoruns, Startup & Registry Execution Paths

```
.\labs\autorun-audit.ps1

reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Autoruns from Sysinternals provides a broad view of startup locations: services, scheduled tasks, Run/RunOnce, Winlogon, image hijacks and many other autostart extensibility points. Use it for discovery, then verify ACL and execution context manually.

Do not "fix" a finding by deleting startup entries during assessment Privilege escalation testing is not system cleanup. Evidence first, minimal proof, then restore exactly what the engagement requires.

9. DLL / Search-Path & Process Behavior

DLL search-order or missing-DLL issues require runtime evidence. Do not infer exploitation only from a directory listing. Process Monitor is ideal for observing file/registry/process activity and filtering on NAME NOT FOUND, process name, path, user and operation.

Safe Procmon workflow

- [] Start capture before launching the authorized application/service.
- [] Filter to the relevant process.
- [] Look for deterministic missing DLL/config/path lookups.
- [] Verify the candidate directory ACL.
- [] Check architecture/signature expectations.
- [] Use training platform for real DLL hijack proof; local module does not drop executable DLL payloads.

Why no DLL payload generator here A general DLL hijack payload crosses quickly into reusable post-exploitation tradecraft. We teach the trust model and evidence collection locally, then use deliberately vulnerable training labs for execution proof.

10. Credential & Configuration Exposure

Sometimes the shortest path to higher privilege is a credential or reusable secret exposed to the current user. Focus on application/service config and approved locations, not indiscriminate credential dumping.

```
.\labs\config-secret-audit.ps1 -Root C:\ProgramData

Get-ChildItem C:\ProgramData -Recurse -File -ErrorAction SilentlyContinue |
Where-Object { $_.Extension -in ".ini",".config",".xml",".json",".yaml" } |
Select-Object -First 100 FullName
```

Protect raw secrets If a real engagement exposes a credential, do not paste plaintext into notes, screenshots, Git, chat, or report. Record masked identifier/location/privilege and store raw evidence only when required by engagement handling rules.

11. Local Reversible Lab: Writable Privileged Script

Local lab models the essence of many Windows privesc paths without registering a real vulnerable service/task. An administrator creates a protected flag and a task script writable by normal users. The administrator must manually invoke the elevated runner; this simulates a privileged service/task consuming attacker-controlled content.

```
# Elevated PowerShell - setup once
.\labs\lab-sim.ps1 -Action setup
```

```
# Normal PowerShell - inspect
.\labs\lab-sim.ps1 -Action status
Get-Content C:\ProgramData\RedTeamLab17\task.ps1
icacls C:\ProgramData\RedTeamLab17\task.ps1
```

Controlled proof

As the normal user, modify only `task.ps1` so it copies the protected flag into `student-output.txt`. Do not create users, services, scheduled tasks, startup persistence or outbound connections.

```
@'
Get-Content 'C:\ProgramData\RedTeamLab17\proof\admin-flag.txt' |
Set-Content -Encoding UTF8 'C:\ProgramData\RedTeamLab17\student-output.txt'
'@ | Set-Content -Encoding UTF8 C:\ProgramData\RedTeamLab17\task.ps1

# In the elevated instructor terminal only:
.\labs\lab-sim.ps1 -Action run-elevated

# Back as normal user:
Get-Content C:\ProgramData\RedTeamLab17\student-output.txt
```

What this proves The privileged executor trusted a script writable by a lower-privilege principal. The exploit primitive is writable privileged input; the elevated runner is only a deterministic lab trigger.

12. Service/Task Misconfiguration Reasoning Template

Step	Evidence
1. Privileged consumer	Service/task/autorun process and account/integrity.
2. Controlled input	Writable binary/script/config/registry key/directory or service object.
3. Trigger	Restart, task schedule, login, application launch or manual lab runner.
4. Minimum proof	Read protected flag or create one harmless privileged artifact.
5. Cleanup	Restore original file/ACL/config and remove artifact.
6. Remediation	Least privilege, restrictive ACL, absolute paths, protected deployment location, correct service/task ACL.

13. Sysinternals Workflow

Tool	Use in Module 17
AccessChk	Enumerate effective access to files, registry keys, processes and service objects.
Autoruns / Autorunsc	Map startup and autostart execution surfaces.
Process Monitor	Observe real-time file, registry and process activity; validate search-path assumptions.
Process Explorer	Inspect process tree, image path, account and integrity context.
Sigcheck	Inspect signatures/hashes of suspicious third-party binaries before trust decisions.

Prefer official Sysinternals Download from Microsoft Learn/Microsoft Store/Sysinternals Live. Avoid repackaged mirrors, especially on a Red Team workstation where tooling supply-chain risk is real.

14. WinPEAS: Automation After Manual Enumeration

WinPEAS from PEASS-ng is useful for breadth, but its output is a lead generator. Run it only in authorized labs, obtain it from the official PEASS-ng release/source, and compare results with your manual notes.

Compare, do not outsource reasoning

Manual finding	Automation question
Token privilege	Did WinPEAS flag it? Is it actually usable in this context?
Service path/ACL	Does it identify same service? Did manual ACL verification agree?
Scheduled task	Does it show privileged task/action? Is target path writable?
Registry/autorun	Did it flag a weak registry or executable ACL?
Credentials/config	Does it expose sensitive content? Was collection necessary and approved?

Tool output is not report text Never paste hundreds of WinPEAS lines into a report. Extract only the evidence that supports a validated attack path and explain why it matters.

15. Kernel / Driver Exploitation: Last Resort

Windows kernel/driver LPE is high-risk: build-specific, mitigation-sensitive and potentially crash-prone. Treat it like Linux kernel exploitation: precise advisory matching, snapshot/restore, RoE approval and source audit before any PoC.

- ☐ Exact Windows build and patch level?
- ☐ Driver/software version affected?
- ☐ Architecture matches?
- ☐ Vendor advisory/CVE confirms applicability?
- ☐ Snapshot exists?
- ☐ Exploit source audited?
- ☐ Less risky configuration path exhausted?
- ☐ RoE permits kernel/driver exploitation?

Version-only conclusions are weak Security updates, backports, Defender/WDAC/HVCI, driver blocklists and configuration can change exploitability. Validate exact conditions; never equate product/version string with guaranteed LPE.

16. Detection & Blue-Team Telemetry

Good Red Team practice asks what defenders can see. Event availability depends on audit policy and EDR configuration, so treat IDs below as telemetry candidates, not guarantees.

Signal	Typical evidence
Process creation	4688 when process-creation auditing is enabled; EDR process tree/command line.
Special privileges	4672 for certain special privileges at logon.
Service changes	Service Control Manager / System logs; 7045 commonly records service installation.
Scheduled tasks	4698/4702 when relevant advanced audit policy is enabled.
File/registry access	Object access events such as 4656/4663 only when auditing/SACL is configured.
Execution path behavior	Procmon/EDR file and registry events; image load and parent/child relationships.

Stealth is not the objective here Module 17 optimizes for understanding, reproducibility and remediation. Evasion belongs later and only after you understand the telemetry you are trying to avoid.

17. Cleanup, Evidence & Retest

```
# Hash evidence
.\labs\evidence-manifest.ps1

# Remove local simulation - elevated PowerShell
.\labs\lab-sim.ps1 -Action cleanup
```

- ☐ Restore modified training script/config.
- ☐ Remove proof artifacts.
- ☐ Do not leave new local users/services/tasks/autoruns.
- ☐ Hash evidence and note timestamps.
- ☐ Retest the remediated ACL/path if you changed it.
- ☐ Document anything that could not be restored automatically.

18. Guided Labs (15)

Lab	Objective
01 - Workspace	Create structured notes/evidence/findings folders.
02 - Token report	Capture user SID, groups, privileges and integrity context.
03 - Manual inventory	Collect OS/users/processes/services/tasks/network baseline.
04 - ACL reading	Compare Get-Acl and icacls on selected lab paths.
05 - Service inventory	Map service account and executable paths.
06 - Task inventory	Map task principal/run level/action.
07 - Autoruns	Review Run/RunOnce and Sysinternals Autoruns output.
08 - AccessChk	Validate effective file/service rights.
09 - Procmon	Validate one file/path lookup hypothesis.
10 - Config exposure	Find candidate config files without dumping secret values.

11 - Local trust lab	Set up reversible writable privileged script simulation.
12 - Controlled proof	Read protected lab flag through privileged runner.
13 - Automation compare	Compare manual notes with WinPEAS.
14 - Evidence & cleanup	Hash evidence and remove simulation.
15 - Finding write-up	Write root cause, impact, remediation and retest evidence.

19. Capstone: Unknown Windows PrivEsc Assessment

Use a disposable Windows training VM/box. Do not inspect walkthrough/source before your first pass. Goal: identify one validated path from current context to higher privilege or prove that no path was validated within the timebox.

Rules

- [] 60-90 minute timebox.
- [] Manual enumeration first 20 minutes.
- [] At least two hypotheses documented before automation.
- [] No credential dumping unless the platform scenario specifically requires it.
- [] No persistence; no external callback infrastructure.
- [] Minimum proof only.
- [] Cleanup + remediation note required.

Deliverables

- [] Token/identity baseline.
- [] Service/task/ACL inventory notes.
- [] Validated or rejected hypotheses with evidence.
- [] One finding or a defensible no-finding conclusion.
- [] Detection/telemetry notes.
- [] Cleanup and retest record.

20. Knowledge Check

1. Apa beda user/group SID, token privilege, dan integrity level?
2. Kenapa medium-integrity administrator process belum sama dengan elevated process?
3. Kenapa unquoted service path tidak otomatis exploitable?
4. Apa arti SERVICE_CHANGE_CONFIG dari sisi security?
5. Kenapa writable service executable lebih penting daripada service name yang terlihat aneh?
6. Apa yang harus dicek pada scheduled task selain account yang menjalankannya?
7. Apa beda WRITE_DAC, WRITE_OWNER dan Modify pada analisis ACL?
8. Kenapa parent directory ACL penting?
9. Kapan Procmon lebih berguna daripada static enumeration?
10. Kenapa enabled SelmpersonatePrivilege bukan finding otomatis?
11. Bagaimana membedakan config credential exposure dari credential dumping?
12. Kenapa WinPEAS harus dijalankan setelah manual enumeration?
13. Apa minimum sufficient proof untuk writable privileged script?
14. Kenapa kernel/driver LPE harus menjadi last resort?
15. Telemetry apa yang bisa muncul ketika service/task/process berubah?
16. Bagaimana remediation untuk weak privileged execution path?
17. Apa yang harus dibersihkan setelah privesc test?
18. Mengapa UAC bukan pengganti authorization/least privilege?
19. Bagaimana membuktikan finding tetap fixed setelah remediation?
20. Kapan sebaiknya lo berhenti dan eskalasi ke engagement lead?

21. Cheat Sheet

```
whoami /all
whoami /priv
Get-ComputerInfo
Get-LocalUser
Get-LocalGroupMember Administrators
Get-CimInstance Win32_Service | select Name,StartName,State,PathName
Get-ScheduledTask
Get-Acl <path> | fl
icacls <path>
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
Get-NetTCPConnection -State Listen
```

Reasoning loop Identity -> privileged executor -> attacker-controlled input -> trigger -> minimum proof -> telemetry -> cleanup ->

remediation -> retest.

22. References & Further Reading

Source	URL
Microsoft - Access Tokens	https://learn.microsoft.com/en-us/windows/win32/secauthz/access-tokens
Microsoft - Security Identifiers	https://learn.microsoft.com/en-us/windows/win32/secauthz/security-identifiers
Microsoft - Mandatory Integrity Control	https://learn.microsoft.com/en-us/windows/win32/secauthz/mandatory-integrity-control
Microsoft - DACLS and ACEs	https://learn.microsoft.com/en-us/windows/win32/secauthz/dacLS-and-aces
Microsoft - icacLS	https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/icacLS
Microsoft - Service Security and Access Rights	https://learn.microsoft.com/en-us/windows/win32/services/service-security-and-access-rights
Microsoft - Security Contexts for Tasks	https://learn.microsoft.com/en-us/windows/win32/taskschd/security-contexts-for-running-tasks
Sysinternals - AccessChk	https://learn.microsoft.com/en-us/sysinternals/downloads/accesschk
Sysinternals - Autoruns	https://learn.microsoft.com/en-us/sysinternals/downloads/autoruns
Sysinternals - Process Monitor	https://learn.microsoft.com/en-us/sysinternals/downloads/procmon
Sysinternals Suite	https://learn.microsoft.com/en-us/sysinternals/downloads/sysinternals-suite
PEASS-ng / WinPEAS releases	https://github.com/peass-ng/PEASS-ng/releases
Omarchy - Windows VM	https://omarchy.org/manual/windows-vm/

Next Module 18 akan masuk Active Directory Fundamentals: domain, forest, DC, OU, users/groups/computers, LDAP, DNS, SMB, Kerberos, NTLM, SPN, GPO, ACL dan trust - sebelum kita melakukan AD enumeration/attack paths.

23. Practical Reference Card

If you find...	Validate next	Typical remediation
Writable privileged executable/script	Owner, DACL, parent directory, privileged consumer, trigger	Move to protected location; restrict DACL; deploy as admin-owned immutable artifact.
Weak service object ACL	Confirm SERVICE_CHANGE_CONFIG/WRITE_DAC and service account	Restrict service DACL to administrators/service-management principals.
Unquoted service path	Enumerate candidate prefixes + write permissions + restart trigger	Quote ImagePath; install under protected directory; remove writable path components.
Privileged scheduled task	Inspect principal/run level/action/arguments and ACLs	Protect action/config; absolute paths; least-privilege task principal.
Weak autorun / registry path	Confirm who writes key/value and target binary path	Restrict registry/file ACL; sign/verify deployment; minimize autorun surface.
Sensitive token privilege	Confirm privilege usability and required local conditions	Remove unnecessary privilege; use constrained service account; harden service design.
Credential in config	Confirm scope/privilege without reproducing secret broadly	Rotate secret; secret manager; least privilege; restrictive file/registry ACL.
DLL/search-path hypothesis	Procmon evidence + missing lookup + writable candidate path	Use absolute paths; safe DLL search APIs; protected directories; code signing/WDAC where appropriate.

Fast lab command sequence

```
Set-ExecutionPolicy -Scope Process Bypass
.\labs\workspace-init.ps1
.\labs\token-report.ps1
.\labs>manual-enum.ps1
.\labs\service-audit.ps1
.\labs\task-audit.ps1
.\labs\autorun-audit.ps1
# Elevated only for local simulation setup/cleanup:
.\labs\lab-sim.ps1 -Action setup
.\labs\lab-sim.ps1 -Action cleanup
.\labs\evidence-manifest.ps1
```

Exit criteria Lo siap lanjut ketika dapat menjelaskan satu Windows privesc path dari token + ACL + privileged consumer + trigger sampai remediation, tanpa bergantung pada satu-click enumerator.